LECTURE SERIES · DATABASE SYSTEMS

SQL Advanced

Aggregation, Subqueries & Programmable Databases

Md. Rakib Trofder

Lecturer, Bangladesh Army University of
Science & Technology



Group By · Having · Subquery · Union · Procedures · Functions · Triggers · Views

SUMMARIZING DATA

GROUP BY — Aggregate Rows

GROUP BY collapses rows sharing a value into one summary row — then aggregate functions compute over each group.

`COUNT()` `SUM()` `AVG()` `MAX()` `MIN()`

groupby.sql

```sql
SELECT dept, AVG(cgpa) AS avg_cgpa,
       COUNT(*) AS students
FROM Students
GROUP BY dept;
```

Rule: every non-aggregated column must be in GROUP BY.

One row per department:

dept	avg_cgpa	students
CSE	3.72	120
EEE	3.58	95
CE	3.61	80

many rows → one summary row per group

Think of it as: sort rows into buckets by dept, then compute one number per bucket.

Order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

WHERE FOR AGGREGATES

HAVING — Filter Groups

WHERE filters rows before grouping. HAVING filters the groups after aggregation — so it can use aggregate functions.

having.sql

```sql
SELECT dept, AVG(cgpa) AS avg_cgpa
FROM Students
WHERE cgpa > 2.0          -- rows first
GROUP BY dept
HAVING AVG(cgpa) > 3.6;   -- then groups
```

Result: only departments whose average CGPA beats 3.6.

WHERE vs HAVING

	WHERE	HAVING
Filters	individual rows	whole groups
Runs	before GROUP BY	after GROUP BY
Aggregates	✗ not allowed	✓ allowed
Example	`cgpa > 2.0`	`AVG(cgpa) > 3.6`

A QUERY INSIDE A QUERY

Subqueries

A subquery runs first and hands its result to the outer query — letting one question depend on the answer to another.

subquery.sql

```sql
-- students above the average
SELECT name, cgpa FROM Students
WHERE cgpa > (
    SELECT AVG(cgpa) FROM Students
);
```

Inner first: AVG runs once, its value feeds the outer WHERE.

WHERE SUBQUERIES APPEAR

In WHERE — compare against a computed value or list

In FROM — treat a result as a temporary table

In SELECT — compute one value per output row

USEFUL OPERATORS

`IN` `NOT IN` `EXISTS` `ANY` `ALL`

COMBINING QUERY OUTPUTS

UNION — Stack Results

UNION appends the rows of two SELECTs into one result. Both queries must return the same number of compatible columns.

union.sql

```sql
SELECT name FROM Students
UNION
SELECT name FROM Alumni;
```

Current + former students in one list, duplicates removed.

UNION vs UNION ALL

UNION removes duplicate rows (slower — must sort/compare).

UNION ALL keeps every row, duplicates included (faster).

Related: INTERSECT and EXCEPT work the same way for ∩ and −.

Note: this is relational algebra's ∪ operator, live in SQL.

SAVED PROGRAMS IN THE DATABASE

Stored Procedures

A stored procedure is a named block of SQL saved inside the database — call it by name to run multi-step logic with parameters.

Reusable — write once, call anywhere

Fast — precompiled, less network traffic

Secure — grant access to the proc, not tables

procedure.sql

```
CREATE OR REPLACE PROCEDURE AddStudent(
  p_name VARCHAR, p_dept VARCHAR, p_cgpa NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
  INSERT INTO Students(name, dept, cgpa)
  VALUES (p_name, p_dept, p_cgpa);
END;
$$;

CALL AddStudent('Rahim', 'CSE', 3.85);  -- run it
```

CALL runs the procedure. Need rows back? In PostgreSQL use a FUNCTION … RETURNS TABLE.

COMPUTATIONS THAT RETURN A VALUE

Functions

A user-defined function computes and RETURNS a single value — so unlike a procedure, it can sit inside a SELECT.

function.sql

```
CREATE OR REPLACE FUNCTION Grade(c NUMERIC)
RETURNS VARCHAR(2)
LANGUAGE plpgsql
AS $$
BEGIN
  IF c >= 3.75 THEN RETURN 'A+';
  ELSE RETURN 'A';
  END IF;
END;
$$;

SELECT name, Grade(cgpa) FROM Students;
```

PROCEDURE vs FUNCTION

	PROCEDURE	FUNCTION
Returns	anything / nothing	exactly one value
Called by	CALL statement	inside SELECT / WHERE
Can modify data	✓ yes	usually read-only
Best for	multi-step operations	reusable calculations

AUTOMATIC REACTIONS TO CHANGES

Triggers

A trigger is code the database fires by itself when an INSERT, UPDATE or DELETE happens — no one calls it explicitly.

WHEN × WHAT

BEFORE | AFTER × INSERT | UPDATE | DELETE

trigger.sql

```
CREATE FUNCTION log_student()  -- step 1
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO AuditLog
  VALUES (NEW.id, 'added', NOW());
  RETURN NEW;
END; $$;

CREATE TRIGGER log_new_student -- step 2
AFTER INSERT ON Students
FOR EACH ROW
EXECUTE FUNCTION log_student();
```

TYPICAL USES

- **Audit logging** — record who changed what, and when
- **Enforce rules** — block or fix invalid changes beyond constraints
- **Sync derived data** — keep totals & counters up to date automatically

NEW / OLD: inside a trigger, NEW and OLD reference the row being inserted, updated or deleted.

SAVED QUERIES AS VIRTUAL TABLES

Views

A view is a named SELECT stored in the database. Query it like a table — the underlying query runs fresh each time.

view.sql

```sql
CREATE VIEW CSE_Toppers AS
SELECT name, cgpa FROM Students
WHERE dept = 'CSE' AND cgpa > 3.7;

SELECT * FROM CSE_Toppers;
```

Virtual: no data is copied — always up to date.

WHY USE VIEWS

Simplicity — hide complex joins behind a clean name

Security — expose only chosen columns & rows

Consistency — one definition reused by everyone

Updatable? simple single-table views can accept INSERT/UPDATE; views with joins or aggregates usually cannot.

STORED SNAPSHOTS FOR SPEED

Materialized Views

A materialized view physically stores the query result. Reads become instant — at the cost of refreshing when data changes.

matview.sql

```
CREATE MATERIALIZED VIEW DeptStats AS
SELECT dept, AVG(cgpa) AS avg_cgpa
FROM Students GROUP BY dept;

REFRESH MATERIALIZED VIEW DeptStats;
```

Trade-off: blazing reads vs. possibly stale data.

VIEW vs MATERIALIZED VIEW

	VIEW	MAT. VIEW
Storage	none — virtual	result stored on disk
Freshness	always current	stale until refreshed
Read speed	runs query each time	instant — precomputed
Best for	security, simplicity	dashboards, heavy reports

KEY TAKEAWAYS

Recap

GROUP BY & HAVING

Aggregate rows, then filter the groups

Subqueries

Nest queries inside WHERE, FROM, SELECT

UNION

Stack results; ALL keeps duplicates

Procedures & Functions

Saved logic; functions return a value

Triggers

Auto-fire on INSERT / UPDATE / DELETE

Views & Materialized

Virtual vs stored query results

Thank You

Md. Rakib Trofder
Lecturer, Bangladesh Army University of Science & Technology